

CA1: Dataframe Manipulation with Spotify Data

Introduction

Pandas is an extremely powerful tool to handle large amounts of tabular data. In this compulsory assignment, you will use Pandas to explore one of the TA's personal spotify data in depth. Additional information:

- Feel free to create additional code cells if you feel that one cell per subtask is not sufficient.
- Remember, Pandas uses very efficient code to handle large amounts of data. For-loops are not efficient. If you ever have to use a for-loop to loop over the rows in the DataFrame, you have *probably* done something wrong.
- Label all graphs and charts if applicable.

Task

I typically enjoy indie and rock music. I am a big fan of everything from old-fashioned rock and roll like Led Zeppelin and Jimi Hendrix, to newer indie artists like Joji and Lana Del Rey. This is why my spotify wrapped for 2023 came as quite a surprise:

Now, I'm no hater of pop music, but this was unexpected. For this assignment, you will investigate my listening habits, including a deep dive into my Ariana Grande listening habits, and try to find an answer to why she was my top artist; was there a fault in the spotify algorithm? Am I actually secretly an *Arianator*? (yes, I did have to look that up). Or am I just lying to myself about how often I listen to guilty pleasure music?

Part 1: Initial loading and exploration

1.0 Import necessary libraries:

pandas, numpy, matplotlib.pyplot (other libraries such as seaborn or plotly are also allowed if you want prettier plots). It might also be a good idea to use **os** for task 2.0

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

1.1 Loading the data

Load the dataset in the file `streaming_history_0.csv` into a Pandas DataFrame called `df_spotify_0`.

```
df_spotify_0 = pd.read_csv('spotify_data/streaminghistory0.csv')
```

1.2 Help function

Use the Python command `help` to help you understand how to use the `pd.DataFrame.head` and `pd.DataFrame.tail` methods.

```
help(pd.DataFrame.head)
```

Help on function head in module pandas.core.generic:

```
head(self, n: 'int' = 5) -> 'Self'  
    Return the first `n` rows.
```

This function returns the first `n` rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it.

For negative values of `n`, this function returns all rows except the last `|n|` rows, equivalent to `df[:n]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

`n` : int, default 5
 Number of rows to select.

Returns

same type as caller
 The first `n` rows of the caller object.

See Also

`DataFrame.tail`: Returns the last `n` rows.

Examples

```
>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon',  
    ...                           'lion',                               'monkey', 'parrot', 'shark', 'whale',  
    ...                           'zebra']})  
>>> df  
   animal  
0  alligator  
1      bee  
2   falcon  
3     lion  
4   monkey  
5   parrot
```

```
6      shark
7      whale
8      zebra
```

Viewing the first 5 lines

```
>>> df.head()
      animal
0  alligator
1       bee
2     falcon
3       lion
4     monkey
```

Viewing the first `n` lines (three in this case)

```
>>> df.head(3)
      animal
0  alligator
1       bee
2     falcon
```

For negative values of `n`

```
>>> df.head(-3)
      animal
0  alligator
1       bee
2     falcon
3       lion
4     monkey
5     parrot
```

`help(pd.DataFrame.tail)`

Help on function tail in module pandas.core.generic:

```
tail(self, n: 'int' = 5) -> 'Self'
    Return the last `n` rows.
```

This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to `df[|n|:]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

`n : int, default 5`
Number of rows to select.

Returns

type of caller
The last ``n`` rows of the caller object.

See Also

`DataFrame.head` : The first ``n`` rows of the caller object.

Examples

```
>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon',  
'lion',  
...                               'monkey', 'parrot', 'shark', 'whale',  
'zebra']})  
>>> df
```

```
   animal  
0  alligator  
1     bee  
2   falcon  
3    lion  
4   monkey  
5   parrot  
6    shark  
7    whale  
8    zebra
```

Viewing the last 5 lines

```
>>> df.tail()  
   animal  
4  monkey  
5  parrot  
6   shark  
7   whale  
8   zebra
```

Viewing the last ``n`` lines (three in this case)

```
>>> df.tail(3)  
   animal  
6  shark  
7  whale
```

```
8 zebra
```

For negative values of `n`

```
>>> df.tail(-3)
      animal
3      lion
4  monkey
5  parrot
6   shark
7   whale
8   zebra
```

1.3 Getting an overview

Print the first **five** and last **ten** rows of the dataframe. Have a quick look at which columns are in the dataset.

```
df_spotify_0.head()
```

	endTime	artistName
trackName \		
0 2022-12-03 02:02	Cigarettes After Sex	
Truly		
1 2022-12-03 02:02	Leonard Cohen	Take This Waltz - Paris
Version		
2 2022-12-06 21:05	Vlad Holiday	So Damn Into
You		
3 2022-12-06 21:05	Lorde	
Team		
4 2022-12-06 21:05	Ariana Grande	Into
You		

	msPlayed
0	30000.0
1	8210.0
2	37895.0
3	8984.0
4	1221.0

```
df_spotify_0.tail(10)
```

	endTime	artistName
trackName \		
11949 2023-01-02 20:58	Ariana Grande	six
thirty		
11950 2023-01-02 20:58	Leonard Cohen	Thanks for the
Dance		
11951 2023-01-02 20:59	Des Rocs	Used to the

Darkness				
11952	2023-01-02	20:59	Caroline Polachek	Hit Me Where It Hurts
11953	2023-01-02	20:59	Caroline Polachek	Hit Me Where It Hurts
11954	2023-01-02	20:59	Kaizers Orchestra	Resistansen
11955	2023-01-02	20:59	Mr.Kitty	After Dark
11956	2023-01-02	20:59	daddy's girl	after dark x sweater weather
11957	2023-01-02	20:59	daddy's girl	after dark x sweater weather
11958	2023-01-02	20:59	daddy's girl	after dark x sweater weather

	msPlayed
11949	1699.0
11950	19483.0
11951	185.0
11952	603.0
11953	208.0
11954	208.0
11955	101447.0
11956	301.0
11957	208.0
11958	789.0

1.4 Formatting correctly

When working with Pandas, it's very useful to have columns which contains dates in a specific format called *datetime*. This allows for efficient manipulation and analysis of time-series data, such as sorting, filtering by date or time, and resampling for different time periods. Figure out which column(s) would be appropriate to convert to datetime, if any, and if so, perform the conversion to the correct format.

```
df_spotify_0['endTime'] = pd.to_datetime(df_spotify_0['endTime'],
format='%Y-%m-%d %H:%M')

start_date = df_spotify_0['endTime'].min()
start_date

Timestamp('2022-12-03 02:02:00')

end_date = df_spotify_0['endTime'].max()
end_date

Timestamp('2023-01-02 20:59:00')
```

1.5 Unique artists

Find how many unique artists are in the dataset.

```
df_spotify_0['artistName'].nunique()  
495
```

1.6 Unique songs

Find how many unique songs are in the dataset.

```
df_spotify_0['trackName'].nunique()  
1308
```

Part 1: Questions

Q1: Which columns are in the dataset?

Q2: What timeframe does the dataset span?

Q3: How many unique artists are in the dataset?

Q4: How many unique songs are in the dataset?

Answer 1: The dataset consists of the columns `endTime`, `artistName`, `trackName` and `msPlayed`.

Answer 2: The dataset span from 2022-12-03 02:02:00 to 2023-01-02 20:59:00.

Answer 3: It is 495 unique artists in the dataset.

Answer 4: It is 1308 unique songs in the dataset.

Part 2: Working with all the data

2.0 Importing all the dataframes

In Task 1, you only worked with about a month worth of data. Now, you will work with over a year worth.

In the *spotify_data* folder, there is more than just one listening record. Load each of the 14 listening records into a dataframe (1 dataframe per listening record), and concatenate them together into one large dataframe named `df`.

```
files = [f"spotify_data/streaminghistory{i}.csv" for i in range(0,14)]  
  
df = pd.concat([pd.read_csv(f) for f in files])  
df.index = [i for i in range(0, len(df))]
```

2.1 Sorting by time

Datasets often aren't perfect. One example of an issue that could occur is that the time-based data might not be in chronological order. If this were to happen, the rows in your dataframe could be in the wrong order. To ensure this isn't an issue in your dataframe, you should sort the dataframe in chronological order, from oldest to newest.

```
df['endTime'] = pd.to_datetime(df['endTime'], format='%Y-%m-%d %H:%M')
df.sort_values(by='endTime')
```

	endTime	artistName \
0	2022-12-03 02:02:00	Cigarettes After Sex
1	2022-12-03 02:02:00	Leonard Cohen
2	2022-12-06 21:05:00	Vlad Holiday
3	2022-12-06 21:05:00	Lorde
4	2022-12-06 21:05:00	Ariana Grande
...	...	...
167435	2023-12-07 21:13:00	Ariana Grande
167437	2023-12-07 21:14:00	Leonard Cohen
167436	2023-12-07 21:14:00	Ariana Grande
167438	2023-12-07 21:17:00	The Vaccines
164529	NaT	The Lumineers

	trackName	msPlayed
0	Truly	30000.0
1	Take This Waltz - Paris Version	8210.0
2	So Damn Into You	37895.0
3	Team	8984.0
4	Into You	1221.0
...	...	...
167435	off the table (with The Weeknd)	13448.0
167437	Thanks for the Dance	9317.0
167436	my hair	23757.0
167438	Your Love Is My Favourite Band	14661.0
164529	Ophelia	371.0

[167439 rows x 4 columns]

2.2 Setting a timeframe

For this investigation, we are only interested in investigating listening patterns from **2023**. Remove any data not from **2023** from the DataFrame.

```
df = df[((df['endTime'] > '2023-01-01 00:00:00') & (df['endTime'] < '2024-01-01 00:00:00'))]
```

2.3 Deleting rows

Often in Data Science, you will encounter when a row entry has the value *NaN*, indicating missing data. These entries can skew your analysis, leading to inaccurate conclusions. For this

task, identify and remove any rows in your DataFrame that contain NaN values. Later in the course, you might encounter other techniques of dealing with missing data, typically referred to as *data imputation*. Here, though, you are just supposed to delete the entire rows with missing data.

```
NaN = df[df.isna().any(axis=1) == True]
df.drop(NaN.index, inplace=True)
```

2.4 Convert from milliseconds to seconds

From `msPlayed`, create a new column `secPlayed` with the data converted from milliseconds to seconds. Then delete the column `msPlayed`.

```
df['secPlayed'] = df['msPlayed'] / 1000
df.drop('msPlayed', axis=1, inplace=True)
df
```

```
-----
-----
KeyError                                Traceback (most recent call
last)
File
/opt/anaconda3/envs/dat200_env/lib/python3.10/site-packages/pandas/
core/indexes/base.py:3802, in Index.get_loc(self, key)
    3801 try:
-> 3802     return self._engine.get_loc(casted_key)
    3803 except KeyError as err:

File index.pyx:153, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:182, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7081, in
pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7089, in
pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'msPlayed'
```

The above exception was the direct cause of the following exception:

```
KeyError                                Traceback (most recent call
last)
Cell In[114], line 1
----> 1 df['secPlayed'] = df['msPlayed'] / 1000
      2 df.drop('msPlayed', axis=1, inplace=True)
      3 df

File
```

```

/opt/anaconda3/envs/dat200_env/lib/python3.10/site-packages/pandas/
core/frame.py:4090, in DataFrame.__getitem__(self, key)
    4088 if self.columns.nlevels > 1:
    4089     return self._getitem_multilevel(key)
-> 4090 indexer = self.columns.get_loc(key)
    4091 if is_integer(indexer):
    4092     indexer = [indexer]

```

File

```

/opt/anaconda3/envs/dat200_env/lib/python3.10/site-packages/pandas/
core/indexes/base.py:3809, in Index.get_loc(self, key)
    3804     if isinstance(casted_key, slice) or (
    3805         isinstance(casted_key, abc.Iterable)
    3806         and any(isinstance(x, slice) for x in casted_key)
    3807     ):
    3808         raise InvalidIndexError(key)
-> 3809     raise KeyError(key) from err
    3810 except TypeError:
    3811     # If we have a listlike key, _check_indexing_error will
raise
    3812     # InvalidIndexError. Otherwise we fall through and re-
raise
    3813     # the TypeError.
    3814     self._check_indexing_error(key)

```

KeyError: 'msPlayed'

2.5 Finding top 10 favorite artists

Find the top `ten` artists with the highest total play time (in seconds). Plot your findings in a bar graph. (hint: start by creating a new DataFrame with only `artistName` and your time column. To proceed, you will also likely need the `groupby` command from Pandas.)

```

artist_play_count = df.groupby(['artistName'], as_index=False)
['secPlayed'].sum()
top_ten = artist_play_count.sort_values(by='secPlayed',
ascending=False).head(10)

plt.figure(figsize=(20, 8))
barplot = sns.barplot(x='artistName', y='secPlayed', data=top_ten,
palette='coolwarm', orient='v')
plt.xlabel('Artist name')
plt.ylabel('Seconds played')
plt.title('Top 10 artists by play count')
plt.show()

```

```

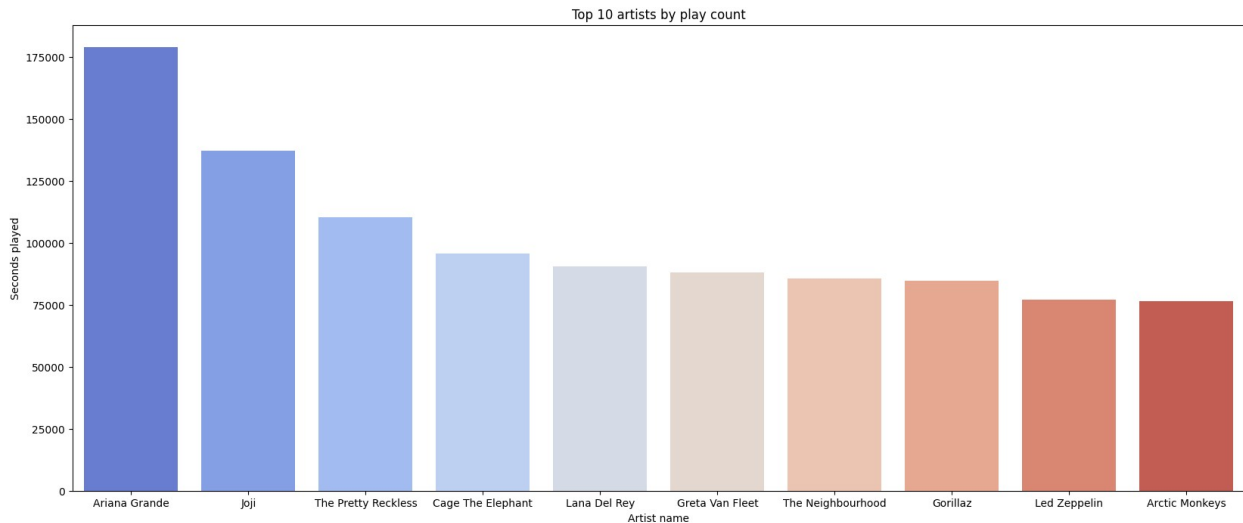
/var/folders/sj/p7jz762n09j7jz2k9h7bd_y80000gn/T/
ipykernel_98588/4152624186.py:5: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be

removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
barplot = sns.barplot(x='artistName', y='secPlayed', data=top_ten,
palette='coolwarm', orient='v')
```



2.6 Finding top 10 favorite songs

Find the top ten songs with the highest play time. Create a graph visualizing the results.

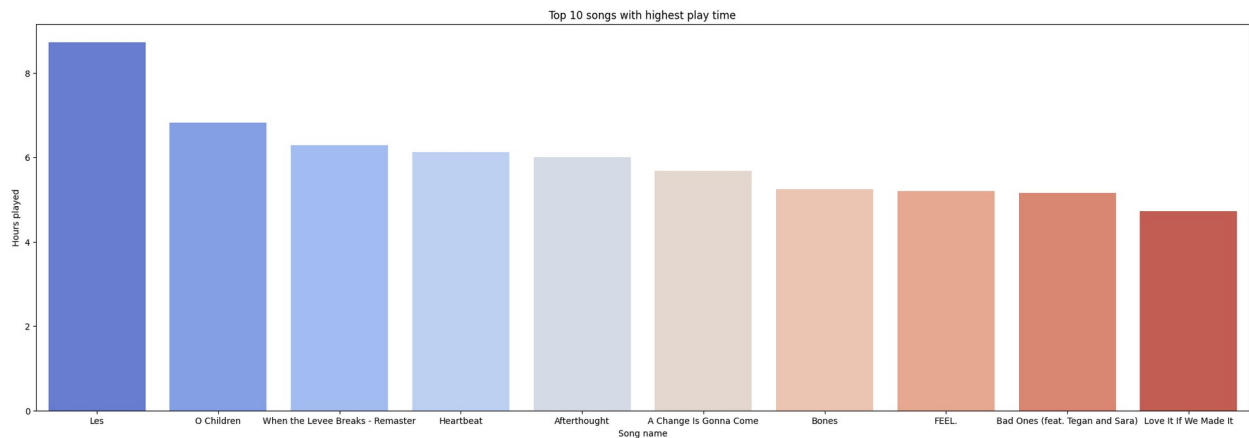
```
top_ten_songs = df.groupby('trackName', as_index=False)
['secPlayed'].sum().sort_values(by='secPlayed',
ascending=False).head(10)
top_ten_songs['hourPlayed'] = top_ten_songs['secPlayed'] / 3600
```

```
plt.figure(figsize=(25, 8))
barplot = sns.barplot(x='trackName', y='hourPlayed',
data=top_ten_songs, palette='coolwarm', orient='v')
plt.xlabel('Song name')
plt.ylabel('Hours played')
plt.title('Top 10 songs with highest play time')
plt.show()
```

```
/var/folders/sj/p7jz762n09j7jz2k9h7bd_y80000gn/T/
ipykernel_98588/1070510806.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
barplot = sns.barplot(x='trackName', y='hourPlayed',
data=top_ten_songs, palette='coolwarm', orient='v')
```



Part 3: Further analysis

3.0 Average listening time by hour

Generate a plot that displays the average amount of time that music is played for each hour of the day.

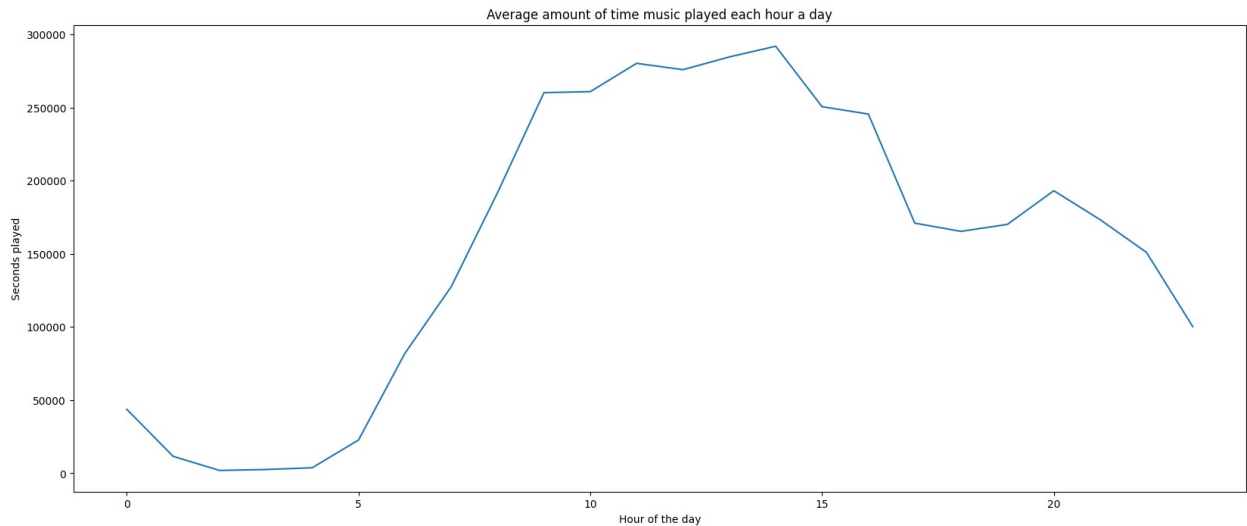
```
new_df = df.copy()

def hour_of_day(x):
    return x.hour

new_df['Hour of day'] = df['endTime'].apply(hour_of_day)
avg_time = new_df.groupby(['Hour of day'], as_index=False)
['secPlayed'].sum()

plt.figure(figsize=(20, 8))
graph = sns.lineplot(x='Hour of day', y='secPlayed', data=avg_time,
palette='coolwarm', orient='x')
plt.xlabel('Hour of the day')
plt.ylabel('Seconds played')
plt.title('Average amount of time music played each hour a day')
plt.show()

/var/folders/sj/p7jz762n09j7jz2k9h7bd_y80000gn/T/
ipykernel_98588/1087231612.py:10: UserWarning: Ignoring `palette`
because no `hue` variable has been assigned.
    graph = sns.lineplot(x='Hour of day', y='secPlayed', data=avg_time,
palette='coolwarm', orient='x')
```



3.1 Morning music and evening music

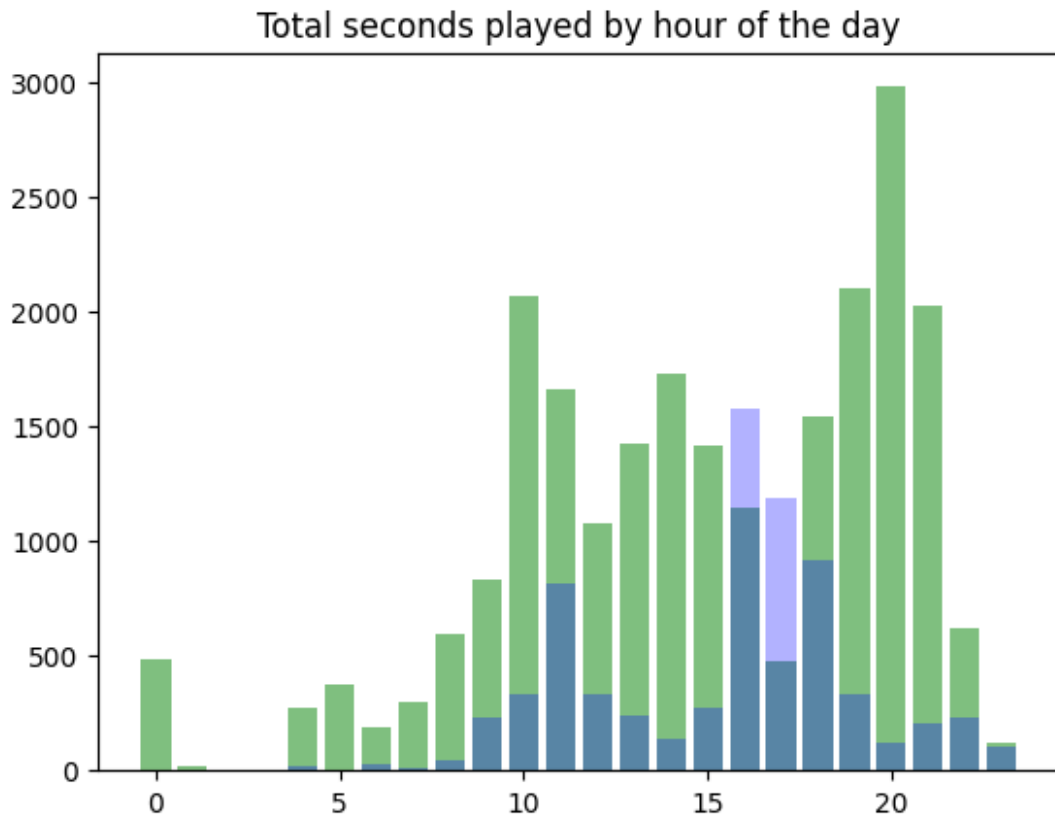
I think many people find that some types of music are more suitable for morning listening and some music is more suitable for evening listening. Create a plot that compares the play time of the artists *Leonard Cohen* and *Rage Against the Machine* on an hour-by-hour basis. See if there are any differences.

```
lc = new_df[new_df['artistName'] == 'Leonard Cohen']
ratm = new_df[new_df['artistName'] == 'Rage Against The Machine']

lc_sorted = lc.groupby(['Hour of day'])['secPlayed'].sum()
ratm_sorted = ratm.groupby(['Hour of day'])['secPlayed'].sum()

plt.bar(lc_sorted.index, lc_sorted.values, color='green', alpha=0.5,
label='Leonard Cohen')
plt.bar(ratm_sorted.index, ratm_sorted.values, color='blue',
alpha=0.3, label='Rage Against The Machine')
plt.title('Total seconds played by hour of the day')

Text(0.5, 1.0, 'Total seconds played by hour of the day')
```



3.2 Analysing skipped songs

Determining whether a song was skipped or listened to can be challenging. For this analysis, we'll simplify by defining a skipped song as any track played for less than 30 seconds. Conversely, a song played for 30 seconds or more is considered listened to. Add a column to your DataFrame to reflect this criteria: set the value to 1 if the song was played for less than 30 seconds (indicating a skipped song), and 0 if it was played for 30 seconds or longer.

```
new_df.loc[new_df['secPlayed'] >= 30, 'Skipped'] = int(0)
new_df.loc[new_df['secPlayed'] < 30, 'Skipped'] = int(1)
new_df
```

	endTime	artistName	
trackName \			
10881	2023-01-01 01:17:00	Ariana Grande	7
rings			
10882	2023-01-01 01:17:00	Ariana Grande	7
rings			
10883	2023-01-01 01:17:00	Ariana Grande	
positions			
10884	2023-01-01 01:17:00	Peach Pit	Being so
Normal			
10885	2023-01-01 01:17:00	Kelly Clarkson	Santa, Can't You
Hear Me			

```

...
...
167434 2023-12-07 21:13:00 Lana Del Rey Art
Deco
167435 2023-12-07 21:13:00 Ariana Grande off the table (with The
Weeknd)
167436 2023-12-07 21:14:00 Ariana Grande my
hair
167437 2023-12-07 21:14:00 Leonard Cohen Thanks for the
Dance
167438 2023-12-07 21:17:00 The Vaccines Your Love Is My Favourite
Band

secPlayed Hour of day Skipped
10881 0.139 1 1.0
10882 0.487 1 1.0
10883 0.417 1 1.0
10884 2.205 1 1.0
10885 0.278 1 1.0
...
167434 38.298 21 0.0
167435 13.448 21 1.0
167436 23.757 21 1.0
167437 9.317 21 1.0
167438 14.661 21 1.0

[156539 rows x 6 columns]

```

3.3 Plotting skipped songs

Create a pie-chart that compares amount of skipped songs to amount of non-skipped songs.

```

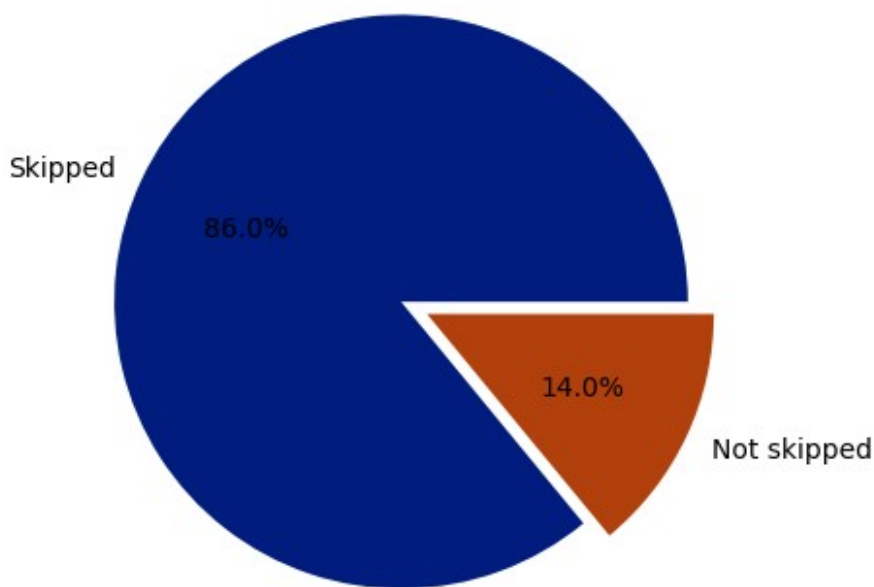
num_skipped = new_df['Skipped'].value_counts()
labels = ['Skipped', 'Not skipped']
explode = [0, 0.1]

palette_color = sns.color_palette('dark')
plt.pie(num_skipped, labels=labels, colors=palette_color,
explode=explode, autopct='%1.1f%%' )
plt.title('Percentage of Skipped vs Not skipped songs')

Text(0.5, 1.0, 'Percentage of Skipped vs Not skipped songs')

```

Percentage of Skipped vs Not skipped songs



3.4 Artists by percentage of songs skipped

For each artist in the dataset, calculate which percentage of their songs was skipped. Store this information in a new DataFrame called `df_skipped`. Store the percentage of skipped songs in a new column named `SkipRate` **Example:** If an artist has **100** songs in your dataset and **25** of these were skipped, the percentage of skipped songs for this artist would be $\frac{25}{100} = 25\%$

```
songs_by_artist = new_df['artistName'].value_counts()
songs_by_artist = pd.DataFrame(songs_by_artist)
songs_by_artist

songs_skipped_by_artist = new_df.groupby(['artistName'],
as_index=False)['Skipped'].sum()
songs_skipped_by_artist.set_index('artistName', inplace=True)

df_skipped = pd.merge(songs_by_artist, songs_skipped_by_artist,
left_index=True, right_index=True)
df_skipped['SkipRate'] = round((df_skipped['Skipped'] /
df_skipped['count']) * 100, 2)
df_skipped
```

	count	Skipped	SkipRate
artistName			
Ariana Grande	19337	19246.0	99.53
Cage The Elephant	4627	4082.0	88.22

Joji	3611	2694.0	74.61
The Pretty Reckless	3278	2570.0	78.40
Lana Del Rey	3028	2480.0	81.90
...	...	...	...
Gloria Gaynor	1	0.0	0.00
eevee	1	1.0	100.00
Beach Fossils	1	1.0	100.00
Idealism	1	1.0	100.00
LoDub	1	1.0	100.00

[956 rows x 3 columns]

3.5 Comparing artists by skip-rate

Find the **three** top artists with the lowest skip-rate and the **three** with the highest. Print their names, along with their skip-rate.

```
lowest_sr = df_skipped.sort_values(by='SkipRate').head(3)
highest_sr = df_skipped.sort_values(by='SkipRate').tail(3)
```

```
print('Top three artists with lowest skip-rate')
for i in range(0, 3):
    print(f"{lowest_sr.iloc[i].name}; skiprate: {round(lowest_sr['SkipRate'].iloc[i], 2)}%")
```

```
print('\nTop three artists with highest skip-rate')
for i in range(0, 3):
    print(f"{highest_sr.iloc[i].name}; skiprate: {round(highest_sr['SkipRate'].iloc[i], 2)}%")
```

```
Top three artists with lowest skip-rate
Gloria Gaynor; skiprate: 0.0%
Roc Boyz; skiprate: 11.11%
LACES; skiprate: 14.29%
```

```
Top three artists with highest skip-rate
Leah Kate; skiprate: 100.0%
Kayra; skiprate: 100.0%
LoDub; skiprate: 100.0%
```

Part 4: God Is a Data Scientist - The Ariana Deep-Dive

4.0 Ariana-DataFrame:

Create a new DataFrame called *df_ariana*, containing only rows with music by Ariana Grande.

```
df_ariana = new_df[new_df['artistName'] == 'Ariana Grande']
df_ariana
```

```

trackName \
10881 2023-01-01 01:17:00 Ariana Grande 7
rings
10882 2023-01-01 01:17:00 Ariana Grande 7
rings
10883 2023-01-01 01:17:00 Ariana Grande
positions
10887 2023-01-01 01:17:00 Ariana Grande Santa
Baby
10888 2023-01-01 01:17:00 Ariana Grande Right There (feat. Big
Sean)
...
...
167415 2023-12-07 17:46:00 Ariana Grande Almost Is Never
Enough
167422 2023-12-07 20:51:00 Ariana Grande
needy
167428 2023-12-07 21:13:00 Ariana Grande pete
davidson
167435 2023-12-07 21:13:00 Ariana Grande off the table (with The
Weeknd)
167436 2023-12-07 21:14:00 Ariana Grande my
hair

secPlayed Hour of day Skipped
10881 0.139 1 1.0
10882 0.487 1 1.0
10883 0.417 1 1.0
10887 12.293 1 1.0
10888 22.929 1 1.0
...
167415 28.483 17 1.0
167422 26.220 20 1.0
167428 0.603 21 1.0
167435 13.448 21 1.0
167436 23.757 21 1.0

[19337 rows x 6 columns]

```

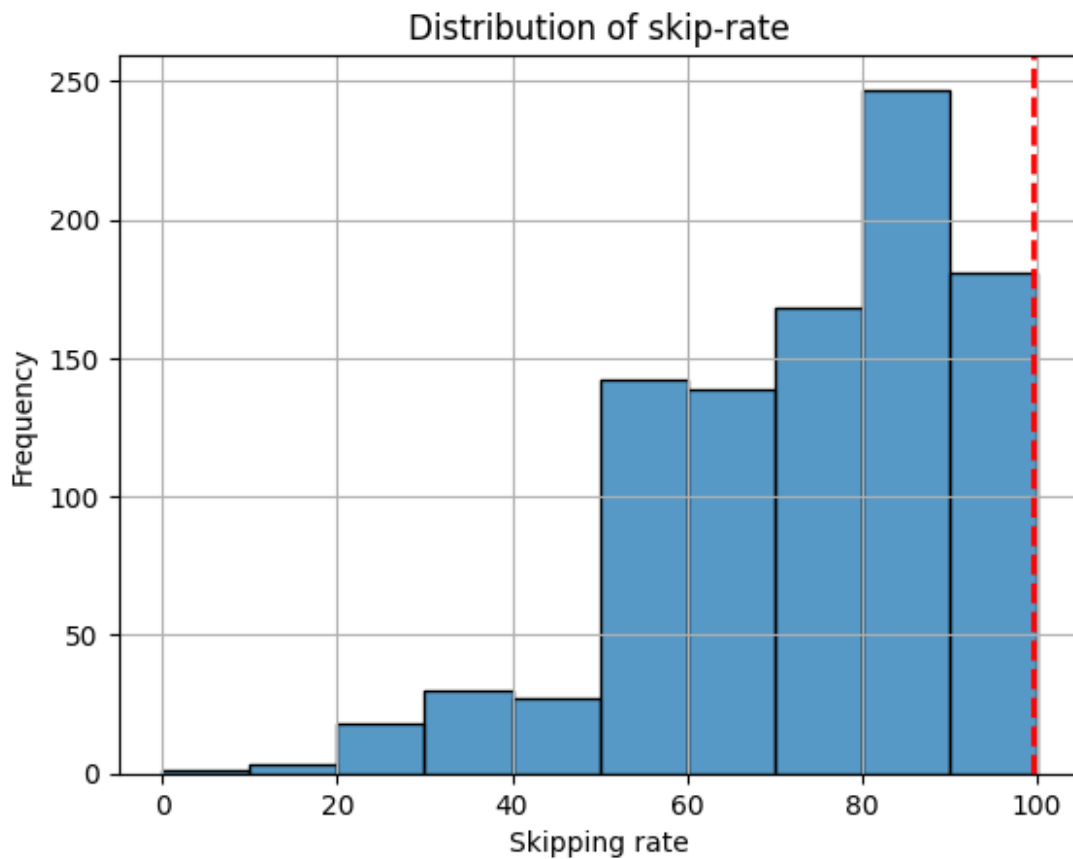
4.1 Average skip rate

Create a histogram of the distribution of the skip-rate values of the different artists in your DataFrame `df_skipped`, with skip rates on one axis and number of artists on the other. Then, retrieve the skip rate for Ariana Grande from your DataFrame `df_skipped`. Run the code in the cell below. Where on this distribution does Ariana Grande fall? Do I skip her songs more than average, or less?

```

histogram = sns.histplot(data=df_skipped, x='SkipRate', bins=10)
plt.xlabel('Skipping rate')
plt.ylabel('Frequency')
plt.title('Distribution of skip-rate')
plt.grid(True)
plt.axvline(df_skipped['SkipRate'].loc['Ariana Grande'], color='r',
linestyle='--', linewidth=2)
plt.show()

```



Part 4: Questions

Q1: Did I skip a lot of Ariana Grande's songs, or did I not, compared to the rest of the dataset?

Q2: What might be some possible reasons for Ariana Grande to be my nr.1 artist?

Answer 1: You did skip a lot of Ariana Grande songs compared to the rest of the dataset.

Answer 2: A possible reason for Ariana Grande being your nr.1 artist is that she has the highest total playtime.